UNIVERSITATEA POLITEHNICĂ TIMIȘOARA
Facultatea de Automatică si Calculatoare
Informatică ID

INGINERIE SOFTWARE

Documentație de proiectare

Sistem portabil de supraveghere a stării de sănătate

Semnături autorizate

Nr. crt.	Nume, prenume	Semnătura
1	Bănăţean Alexandru	
2	Căplescu Adrian	
3	Nica Ionelia-Roxana	
4	Sălăvăstru Iuliana-Adelina	
5	Secetă Alexandru	
6	Seiculescu Ana	
7	Stoicescu Lucian-Mihnea-Doru	
8	Tăşală Alina-Claudia	
9	Tudora Andrei-Petre	
10	Zerfas Răzvan	

Umbrella Corporation

2026

Colectiv de elaborare

Prezentul document a fost elaborat de următoarea echipă de proiectare, în cadrul disciplinei Inginerie Software, anul universitar 2025-2026:

Nr. crt.	Nume și prenume	Rol în proiect	Responsabilități principale
1	Tăsală Alina-Claudia	Programator-șef	contribuție principală la proiectarea tehnică a sistemului, elaborarea și structurarea documentației; contribuții la definirea infrastructurii logice și armonizarea versiunii finale
2	Căplescu Adrian	Ajutor programator-șef	revizuirea și structurarea documentației de proiectare, verificarea coerenței dintre capitolele tehnice, susținerea corelării dintre specificații și soluția propusă și sprijin în armonizarea formei finale a documentului
3	Bănățean Alexandru	Secretar	organizarea materialelor de lucru, centralizarea versiunilor documentelor și susținerea formei unitare a documentației
4	Nica Ionelia-Roxana	Inginer testare	sprijin în formularea scenariilor de funcționare, revizuirea logicii de tratare a situațiilor anormale și verificarea coerenței fluxurilor descrise
5	Sălăvăstru Adelina-Iuliana	Analist de afacere	susținerea formulării contextului de utilizare și a relației dintre funcționalitățile produsului și nevoile utilizatorilor
6	Secetă Alexandru	Specialist marketing	susținerea identității produsului și a modului de prezentare unitară a acestuia în documentație
7	Seiculescu Ana	Dezvoltator backend	contribuții la definirea rolului backend-ului, a fluxului de date și a relației dintre componenta centrală și baza de date
8	Stoicescu Lucian-Mihnea-Doru	Inginer DevOps	contribuție importantă la proiectarea infrastructurii tehnice a sistemului, definirea arhitecturii de deployment, organizarea logică a componentelor
9	Tudora Andrei-Petre	Dezvoltator frontend	sprijin în descrierea interfeței web, a fluxului de navigare și a relației dintre paginile aplicației destinate medicului
10	Zerfas Răzvan	Inginer testare	revizuirea logicii de utilizare și sprijin în verificarea coerenței scenariilor funcționale și a răspunsurilor sistemului

CUPRINS

1. Introducere
2. Arhitectura programului
   2.1. Descriere generală și model arhitectural
   2.2. Mediul de execuție, sistemul de operare și infrastructura logică
   2.3. Componentele principale și tehnologiile utilizate
   2.4. Interacțiunea dintre componente
   2.5. Structura interfeței și fluxul dialogurilor
3. Descrierea componentelor
   3.1. Șablon de descriere unitară a modulelor
   3.2. Modulul inteligent
   3.3. Aplicația mobilă Android
   3.4. Serverul proxy invers
   3.5. Backend-ul
   3.6. Interfața web
   3.7. Baza de date
   3.8. Modulul de management și acces administrativ
4. Descrierea comunicării între module
   4.1. Principii generale de comunicare
   4.2. Tabelul canalelor de comunicație
   4.3. Comunicarea dintre modulul inteligent și aplicația mobilă
   4.4. Comunicarea dintre aplicația mobilă și backend
   4.5. Comunicarea dintre interfața web și backend
   4.6. Comunicarea dintre backend și baza de date
   4.7. Gestionarea erorilor și a răspunsurilor
5. Structuri de baze de date și fișiere
   5.1. Principii de proiectare a bazei de date
   5.2. Schema relațională generală și diagrama de relații între entități
   5.3. Definirea tabelelor
   5.4. Relații, chei, indecși și restricții
   5.5. Fișiere auxiliare și structuri temporare

1. INTRODUCERE

Prezentul document descrie proiectarea tehnică a sistemului MetaHuman și dezvoltă soluția propusă în documentul de Specificații, prin definirea arhitecturii programului, a componentelor sale, a mecanismelor de comunicare și a structurilor de date necesare implementării.

Sistemul proiectat păstrează direcția generală stabilită în documentul anterior: un modul inteligent bazat pe Arduino colectează date fiziologice, le transmite către o aplicație mobilă Android, iar aceasta le redirecționează către un backend central implementat în ASP.NET Core. Datele sunt stocate într-o bază de date PostgreSQL și sunt puse la dispoziția utilizatorului printr-o interfață web realizată în Vue.js. În paralel, infrastructura include un server proxy invers bazat pe Nginx, protecție la nivel de acces prin Cloudflare și un mecanism de administrare securizată bazat pe WireGuard.

În proiectarea de față, sistemul este tratat ca un ansamblu de componente distincte, conectate prin interfețe bine definite. Parametrii monitorizați în varianta curentă a soluției sunt semnalul ECG, pulsul și temperatura. Temperatură înseamnă, în această etapă de proiectare, parametrul colectat cu ajutorul senzorului DS18B20 integrat în prototipul hardware.

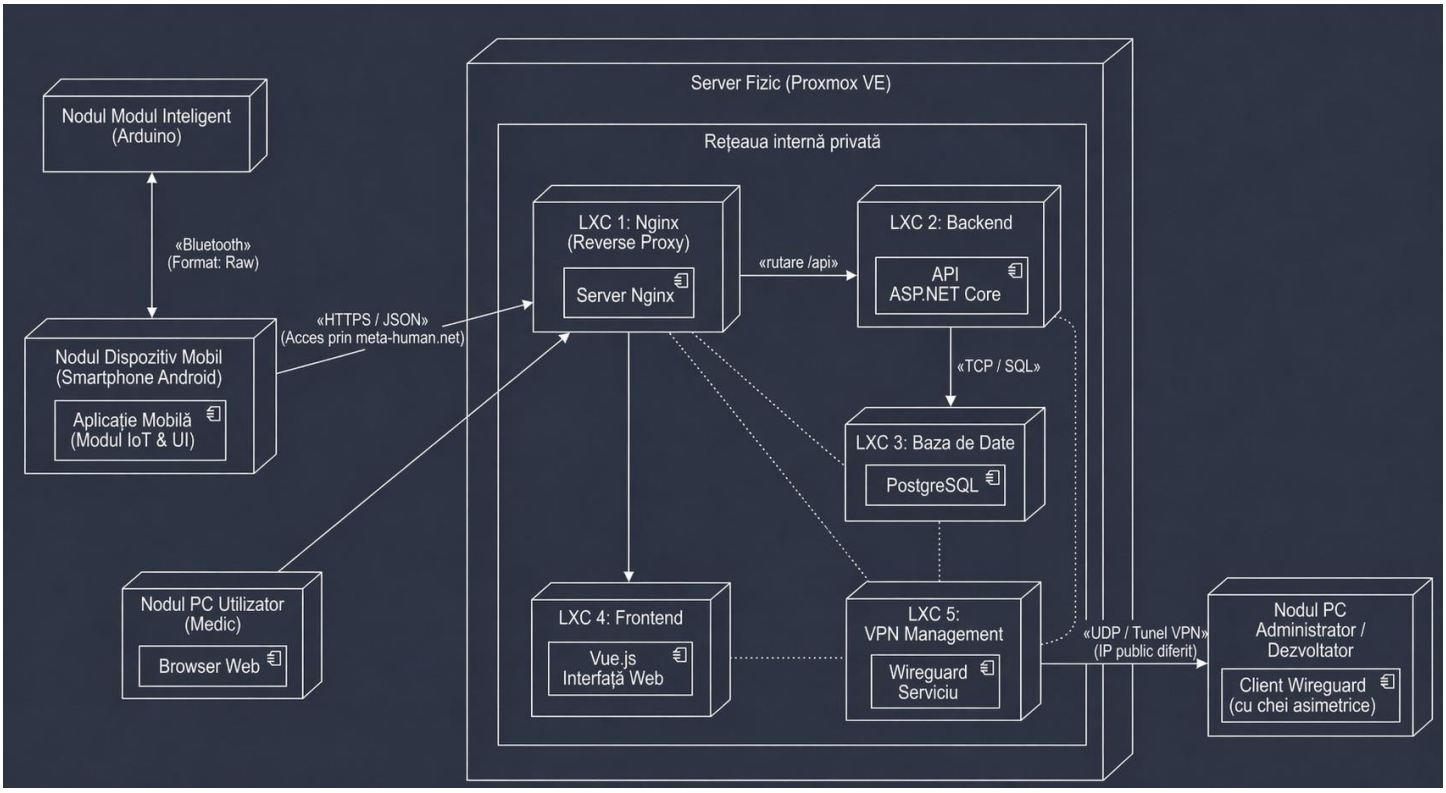
2. ARHITECTURA PROGRAMULUI

2.1. Descriere generală și model arhitectural

Aplicația MetaHuman urmează un model arhitectural de tip client-server distribuit, structurat pe mai multe niveluri funcționale. Alegerea acestei structuri este justificată de nevoia de a separa clar responsabilitățile între colectarea datelor, prezentarea acestora către utilizator, procesarea logicii de business și stocarea persistentă a informațiilor. În plus, sistemul trebuie să poată fi dezvoltat incremental și întreținut în condiții rezonabile, fără ca modificările aduse unei componente să implice rescrierea tuturor celorlalte componente.

Modelul adoptat este unul pe mai multe niveluri, în care se disting următoarele straturi majore:

- stratul de achiziție a datelor, reprezentat de modulul inteligent bazat pe Arduino și senzori;
- stratul de intermediere locală și interfață pacient, reprezentat de aplicația mobilă Android;
- stratul de acces public și rutare, reprezentat de serverul proxy invers;
- stratul de procesare centrală, reprezentat de backend-ul ASP.NET Core;
- stratul de persistență, reprezentat de baza de date PostgreSQL;
- stratul de acces administrativ și mentenanță, reprezentat de infrastructura VPN și virtualizarea logică a serviciilor.



Din punct de vedere al organizării logice, aplicația mobilă și interfața web joacă rolul de clienți ai backend-ului. Backend-ul reprezintă nucleul logic al aplicației și singura componentă care cunoaște în mod direct structura bazei de date și regulile de business. În acest mod se evită dispersarea logicii de validare și interpretare a datelor și se asigură o implementare coerentă a funcționalităților.

2.2. Mediul de execuție, sistemul de operare și infrastructura logică

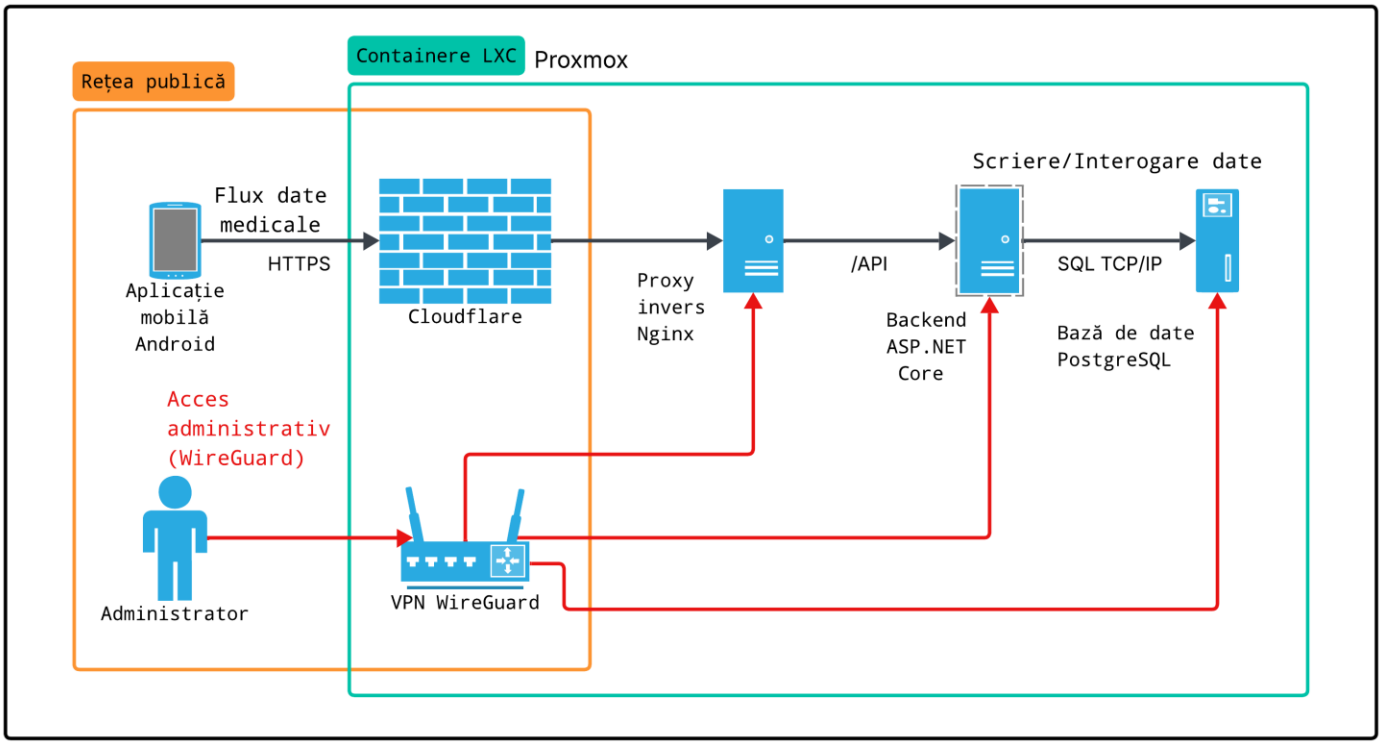
La nivelul infrastructurii centrale, aplicația este proiectată să ruleze pe un nod de virtualizare Proxmox VE, care folosește containere LXC pentru izolarea logică a serviciilor. Alegerea acestei soluții se justifică prin posibilitatea de a separa componentele critice ale sistemului și de a controla mai clar dependențele dintre ele. Fiecare container găzduiește o componentă distinctă, ceea ce simplifică atât administrarea, cât și analiza eventualelor incidente.

În arhitectura propusă există următoarele containere principale:

- un container pentru serverul proxy invers Nginx;
- un container pentru backend-ul ASP.NET Core;
- un container pentru baza de date PostgreSQL;
- un container pentru interfața web;
- un container dedicat accesului administrativ prin WireGuard.

Componentele interne comunică între ele printr-o rețea privată, neexpusă direct către internet. Punctul unic de intrare public în sistem este serverul Nginx. Acesta primește cererile venite de la aplicația mobilă și de la browser-ele utilizate de medici și le direcționează către serviciile interne corespunzătoare. Înainte de a ajunge la

infrastructura centrală, traficul public este filtrat și intermediat prin Cloudflare, care oferă protecție suplimentară și posibilitatea aplicării unor restricții de acces.



Pentru mentenanță și dezvoltare, se utilizează un canal separat de acces, bazat pe WireGuard. Acesta nu participă la fluxul normal al produsului pentru utilizatorii finali, dar asigură un acces controlat și securizat la rețeaua internă a sistemului, fără expunerea directă a serviciilor de bază.

Din punct de vedere al mediului de execuție:

- aplicația mobilă rulează pe Android și este implementată în Java;
- interfața web rulează în browser;
- backend-ul ASP.NET Core rulează pe un sistem de operare Linux găzduit într-un container;
- PostgreSQL rulează de asemenea într-un container izolat;
- Nginx rulează separat și preia rolul de poartă logică către infrastructura internă.

Această arhitectură este compatibilă cu cerințele de securitate, extensibilitate și mentenabilitate formulate anterior, deoarece reduce dependențele directe dintre clienți și straturile de persistență și permite controlul riguros al canalelor de comunicație.

2.3. Componentele principale și tehnologiile utilizate

Sistemul MetaHuman este compus dintr-un ansamblu de module software și componente hardware, fiecare având un rol bine definit. Pentru claritate, relația dintre componente și tehnologiile utilizate este sintetizată în tabelul următor.

Componentă	Tehnologie	Rol principal	Observații de proiectare
Modul inteligent	Arduino UNO, AD8232, DS18B20, HC-05	colectare date fiziologice și transmitere locală	funcționează ca punct de intrare a datelor
Aplicație mobilă	Java Android	interfață pacient și intermediere între hardware și backend	gestionează Bluetooth, stocare temporară și transmitere prin HTTPS
Server proxy invers	Nginx	acces public, rutare și separare trafic	direcționează către frontend și backend
Backend	ASP.NET Core	logica de business și API	componentă centrală, fără stare persistentă locală
Interfață web	Vue.js	interfață pentru medic	utilizată pentru administrare și analiză
Bază de date	PostgreSQL	persistență și integritate relațională	accesibilă exclusiv backend-ului
Protecție acces	Cloudflare	filtrare trafic și protecție	componentă de infrastructură externă
Acces administrativ	WireGuard	administrare securizată	separat de traficul public
Virtualizare	Proxmox VE + LXC	separare logică a serviciilor	suport pentru izolare și întreținere

În proiectarea de față se urmărește ca fiecare tehnologie să fie folosită în zona în care oferă cel mai bun raport între claritate, control și ușurință de implementare. De exemplu, Java Android este aleasă pentru aplicația mobilă deoarece oferă acces direct și matur la funcționalitățile platformei Android, inclusiv la comunicarea Bluetooth și la stocarea locală temporară. Backend-ul ASP.NET Core este ales pentru separarea clară a serviciilor și organizarea eficientă a logicii de business. Vue.js este ales pentru flexibilitatea interfeței web și pentru ușurința construirii unui tablou de bord medical cu grafice și formulare.

2.4. Interacțiunea dintre componente

Fluxul principal de funcționare începe la nivelul modulului inteligent, unde senzorii preiau semnale fiziologice de la pacient. Datele sunt citite de placa Arduino și transmise prin Bluetooth către aplicația mobilă. Aplicația mobilă realizează o validare locală elementară, transformă datele într-o structură JSON și le trimite printr-o conexiune HTTPS către infrastructura centrală. Cererile sunt recepționate de Nginx, care le direcționează către backend-ul ASP.NET Core. Backend-ul validează cererea, verifică identitatea utilizatorului, procesează datele și le salvează în PostgreSQL. În sens invers, atunci când medicul solicită vizualizarea fișei unui pacient sau a istoricului măsurătorilor, interfața web trimite o cerere către backend, care extrage datele necesare din baza de date și le returnează pentru afișare.

În cazul generării alertelor, fluxul se extinde cu un pas suplimentar: după salvarea unei măsurători, backend-ul citește pragurile configurate pentru pacientul respectiv și verifică dacă valoarea observată se află în afara limitelor permise. O alertă nu este generată imediat la prima abatere, ci doar după verificarea persistenței

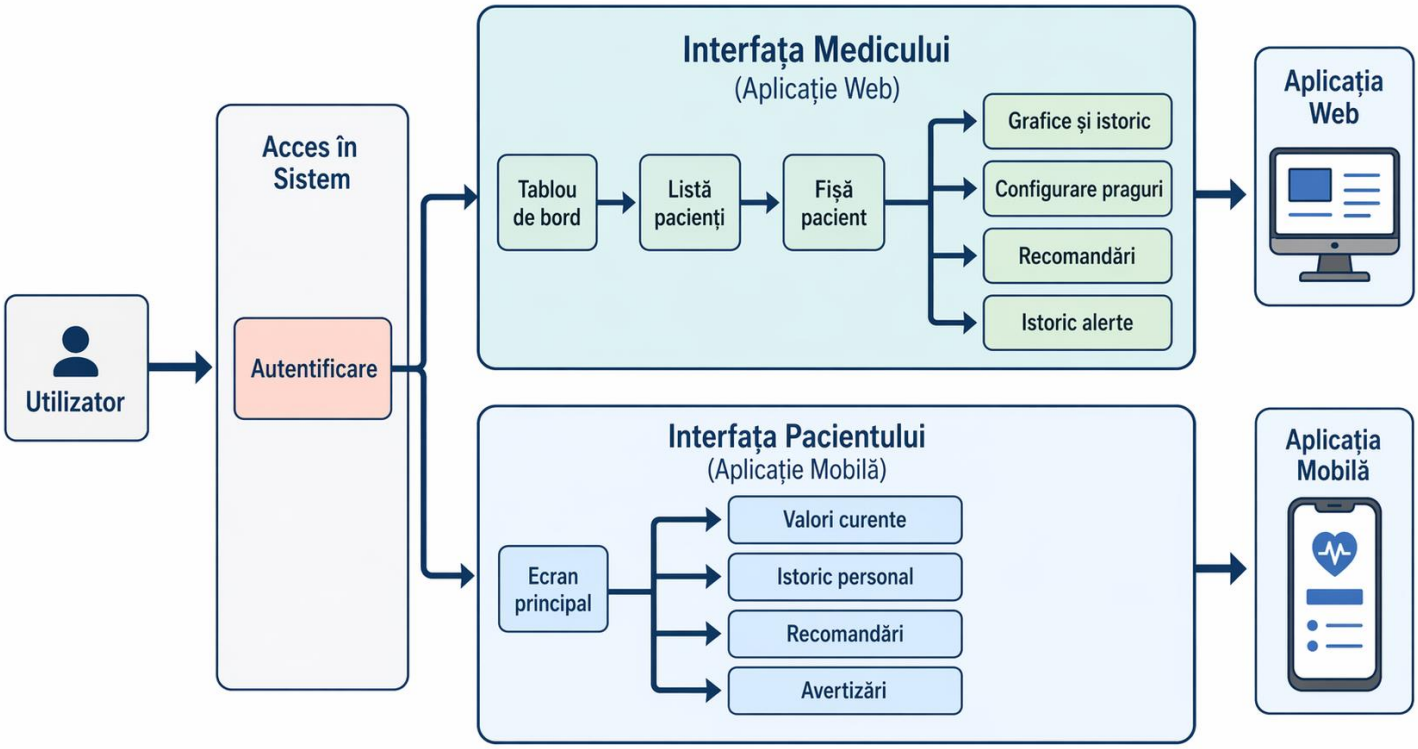
acesteia pe o durată minimă configurabilă. Această decizie de proiectare reduce riscul de alarme false și oferă un comportament mai stabil al sistemului.

2.5. Structura interfeței și fluxul dialogurilor

Sistemul include două tipuri principale de interfețe: interfața pacientului, disponibilă în aplicația mobilă, și interfața medicului, disponibilă în aplicația web. În ambele cazuri, fluxul dialogurilor pornește de la autentificare, însă traseele ulterioare diferă în funcție de rol.

Pentru medic, fluxul general este:
Autentificare → Tablou de bord → Listă pacienți → Fișă pacient → Grafice și istoric / Configurare praguri / Recomandări / Istoric alerte

Pentru pacient, fluxul general este:
Autentificare → Ecran principal → Valori curente / Istoric personal / Recomandări / Avertizări



Interfața medicului trebuie să favorizeze analiza și administrarea, iar interfața pacientului trebuie să favorizeze claritatea și simplitatea.

3. DESCRIEREA COMPONENTELOR

3.1. Șablon de descriere unitară a modulelor

În cadrul prezentei documentații, toate componentele sistemului sunt descrise după aceeași structură, pentru a păstra coerența tehnică și pentru a permite delimitarea clară a responsabilităților în cadrul echipei. Fiecare modul este prezentat din perspectiva rolului său în arhitectura generală, a intrărilor pe care le primește, a prelucrărilor pe care le efectuează, a ieșirilor pe care le produce și a interacțiunilor pe care le are cu celelalte module ale aplicației. Această abordare răspunde direct cerinței profesorului ca proiectarea să permită atât împărțirea exactă a muncii în cadrul echipei, cât și trecerea firească la etapa de codificare.

În plus, acolo unde este necesar, descrierea modulelor este completată prin observații de proiectare, cu rolul de a preciza alegerile tehnice și limitele de responsabilitate ale fiecărei componente. Astfel se evită suprapunerea logicii între componente și se clarifică locul în care sunt implementate validarea, autentificarea, generarea alertelor sau stocarea datelor.

3.2. Modulul inteligent

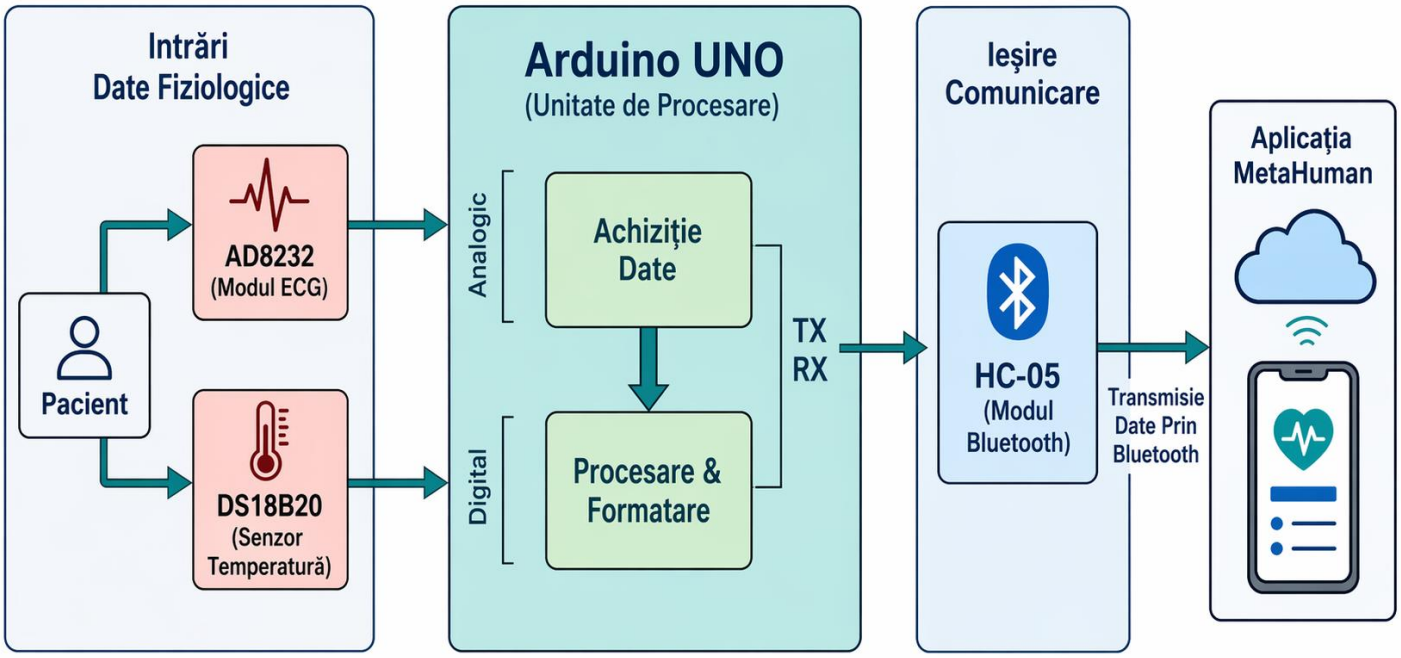
Modulul inteligent reprezintă componenta hardware a sistemului și are rolul de a colecta date fiziologice de la pacient, constituind punctul de intrare al informației în cadrul aplicației MetaHuman. În varianta curentă a prototipului, acest modul este construit pe o placă Arduino UNO, la care sunt conectate un modul AD8232 pentru captarea semnalului ECG, un senzor DS18B20 pentru temperatură și un modul HC-05 pentru comunicarea prin Bluetooth. Alegerea acestor componente se bazează pe caracterul lor accesibil, pe ușurința integrării și pe faptul că permit realizarea unui prototip funcțional al fluxului de monitorizare stabilit în documentul de Specificații.

Intrările modulului inteligent sunt semnalele provenite de la senzorii conectați. În cazul AD8232, intrarea constă în semnalul electric preluat de la electrozi și convertit într-o formă care poate fi citită de microcontroler. În cazul DS18B20, intrarea este valoarea temperaturii măsurate prin contact. Aceste informații nu sunt interpretate medical la nivelul plăcii Arduino, ci doar captate și pregătite pentru transmisie.

Prelucrările realizate la acest nivel sunt limitate și deliberate. Modulul inteligent citește periodic valorile disponibile, efectuează conversia analog-digitală acolo unde este necesar, marchează tipul parametrului transmis și compune un flux de date care poate fi preluat de aplicația mobilă. În această etapă, modulul nu implementează logică de business, nu compară valori cu praguri și nu generează alerte. Această alegere permite centralizarea logicii în backend și evită duplicarea regulilor.

Ieșirile modulului inteligent sunt reprezentate de datele brute sau semistructurate transmise prin Bluetooth către aplicația mobilă. Pentru temperatură și puls, ieșirea poate fi o valoare individuală sau o secvență scurtă de valori. Pentru ECG, ieșirea poate fi un șir de eșantioane colectate într-un interval de timp limitat.

Interacțiunile acestui modul sunt strict delimitate. El interacționează direct doar cu senzorii și cu aplicația mobilă Android. Nu are acces la internet, nu comunică direct cu backend-ul și nu interacționează cu baza de date sau cu interfața web. Această izolare simplifică proiectarea și permite înlocuirea sau extinderea hardware-ului fără modificarea profundă a componentelor centrale.



3.3. Aplicația mobilă Android

Aplicația mobilă Android are un rol esențial în arhitectura sistemului, deoarece funcționează atât ca interfață pentru pacient, cât și ca intermediar între modulul inteligent și infrastructura centrală. În proiectarea de față, aplicația mobilă este prevăzută a fi implementată în Java, pe platforma Android, tocmai pentru a permite control direct asupra funcționalităților specifice sistemului de operare, în special asupra comunicării Bluetooth, a stocării locale temporare și a notificărilor. Alegerea acestei tehnologii este coerentă cu scopul aplicației și cu distribuirea muncii în echipă.

Intrările aplicației mobile sunt de două tipuri. Primul tip este reprezentat de datele primite de la modulul inteligent prin Bluetooth. Al doilea tip constă în interacțiunile pacientului cu aplicația, cum ar fi autentificarea, navigarea între ecrane, solicitarea istoricului sau consultarea recomandărilor și alertelor. În plus, aplicația primește și răspunsurile venite de la backend, pe care trebuie să le interpreteze și să le afișeze într-o formă accesibilă.

Prelucrările efectuate de aplicația mobilă sunt mai bogate decât cele ale modulului inteligent, dar rămân totuși limitate la rolul de client. Aplicația validează local structura datelor primite, împachetează informațiile într-o structură JSON, gestionează sesiunea utilizatorului autentificat și trimite datele către infrastructura centrală prin HTTPS. Dacă rețeaua nu este disponibilă temporar, aplicația poate păstra local datele neexpediate și le poate retransmite ulterior. În plus, ea afișează valorile curente, recomandările, avertizările și alertele și gestionează logica de prezentare specifică rolului pacientului.

Ieșirile aplicației mobile sunt, pe de o parte, cereri securizate către backend, iar pe de altă parte, rezultatele vizuale și notificările prezentate utilizatorului. Aceste ieșiri trebuie să fie coerente și suficient de clare pentru un utilizator care nu are neapărat competențe tehnice sau medicale avansate.

Interacțiunile aplicației mobile sunt multiple. Ea interacționează cu modulul inteligent prin Bluetooth, cu infrastructura centrală prin HTTPS și, indirect, prin backend, cu baza de date. Totodată, ea interacționează vizual cu pacientul, fiind componenta cea mai apropiată de utilizator în fluxul de monitorizare. În proiectarea actuală, aplicația mobilă nu conține regulile complete de generare a alertelor și nu are acces direct la structura bazei de date; aceste responsabilități rămân centralizate în backend, ceea ce simplifică întreținerea și menține coerența logicii aplicației.

3.4. Serverul proxy invers

Serverul proxy invers, implementat cu Nginx, reprezintă punctul unic de acces public către serviciile interne ale sistemului. În cadrul arhitecturii MetaHuman, această componentă are rolul de a separa spațiul public de rețeaua internă privată și de a controla modul în care cererile sunt direcționate către componentele interne. Nginx este ales deoarece permite rutarea clară a cererilor, terminarea conexiunilor securizate și o bună separare între traficul destinat interfeței web și traficul destinat backend-ului.

Intrările serverului proxy invers sunt reprezentate de cererile HTTPS venite din aplicația mobilă și din browser-ele utilizate de medici. Aceste cereri ajung la infrastructură după filtrarea și protecția oferite de Cloudflare, care se află în fața punctului de acces public.

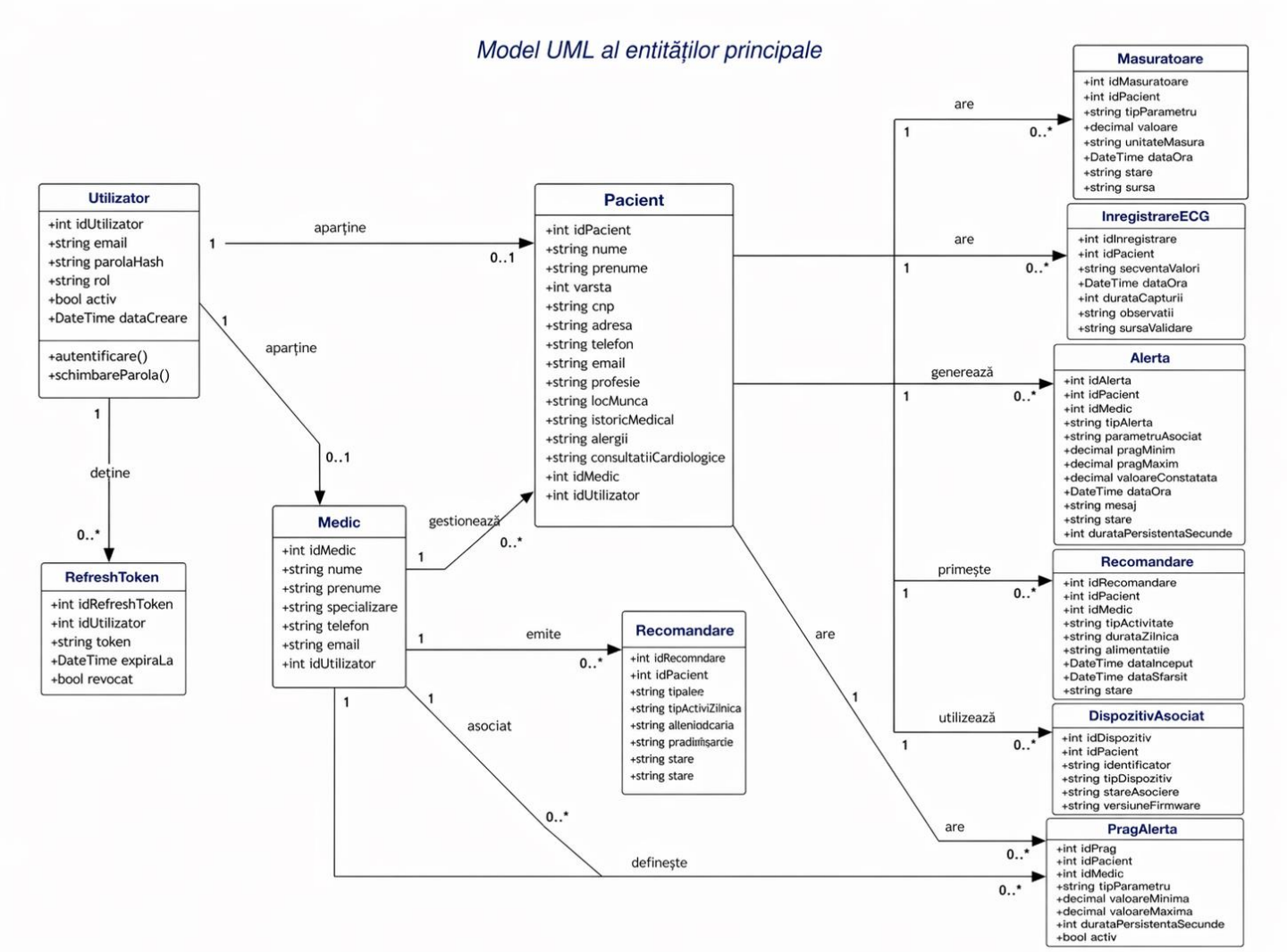
Prelucrările efectuate de Nginx constau în terminarea conexiunilor securizate, identificarea destinației logice a fiecărei cereri și rutarea traficului către containerul intern potrivit. Cererile pentru resursele interfeței web sunt direcționate către frontend, iar cererile către traseele API sunt direcționate către backend. Nginx nu interpretează conținutul medical al datelor și nu implementează logică de business.

Ieșirile serverului proxy invers sunt cererile redirecționate în rețeaua internă, precum și răspunsurile transmise înapoi către clienții externi. Interacțiunile sale sunt clar delimitate: la exterior comunică cu aplicația mobilă și cu browser-ele, iar în interior comunică doar cu frontend-ul și backend-ul.

Din perspectivă de proiectare, folosirea unui server proxy invers reprezintă o soluție sănătoasă, deoarece reduce expunerea directă a componentelor interne și permite adaptarea mai flexibilă a infrastructurii fără schimbarea punctelor de acces utilizate de clienți.

3.5. Backend-ul

Backend-ul este componenta centrală a sistemului și reprezintă nucleul logicii de business. El este proiectat să ruleze ca aplicație ASP.NET Core și să expună puncte de acces API prin care aplicația mobilă și interfața web pot transmite cereri și primi răspunsuri. În arhitectura propusă, backend-ul este singura componentă care are acces direct la baza de date și singura care implementează regulile complete de validare, autorizare, persistare și generare a alertelor.



Intrările backend-ului sunt toate cererile relevante pentru funcționarea sistemului: autentificare medic, autentificare pacient, transmitere măsurători, solicitare istoric, configurare praguri, emitere recomandări, consultare alerte și generare de rapoarte. Aceste cereri ajung la backend prin intermediul Nginx și sunt recepționate în format JSON sau, după caz, în cadrul unui flux de cereri API specifice.

Prelucrările backend-ului sunt cele mai importante din întregul sistem. El validează structura și conținutul cererilor, verifică identitatea și drepturile utilizatorului, salvează măsurătorile, înregistrează capturile ECG, interoghează pragurile configurate pentru fiecare pacient și parametru și evaluează dacă o valoare observată depășește limitele normale. Tot aici este proiectată logica potrivit căreia o alertă nu se generează automat la prima valoare anormală, ci numai dacă abaterea persistă pe o durată minimă configurabilă. În plus, backend-ul gestionează recomandările medicale, istoricul pacientului și compunerea răspunsurilor necesare pentru interfața web și aplicația mobilă.

Ieșirile backend-ului sunt răspunsurile JSON întoarse către clienți și operațiile de citire și scriere efectuate asupra bazei de date. Aceste răspunsuri pot conține confirmări de salvare, date agregate pentru grafice, istorice, alerte sau erori de validare ori autorizare.

Interacțiunile backend-ului sunt bine delimitate. În zona de intrare el comunică cu Nginx și, indirect, cu aplicația mobilă și interfața web. În zona de persistență comunică exclusiv cu PostgreSQL. În proiectarea internă, backend-ul poate fi împărțit logic în controlere, servicii de business, modele de transfer de date și componentă de acces la date, astfel încât responsabilitățile să fie separate și codul să rămână ușor de întreținut.

3.6. Interfața web

Interfața web este destinată în principal medicului și are rolul de a oferi acces la funcțiile de administrare și analiză ale sistemului. Ea este proiectată în Vue.js și funcționează ca un client al backend-ului, fără a conține logică de business esențială sau acces direct la baza de date. Alegerea unei interfețe web separate pentru medic răspunde nevoii de a avea un mediu de lucru mai bogat, mai potrivit pentru afișarea simultană a listelor, graficelor și formularelor.

Intrările interfeței web sunt reprezentate de acțiunile medicului și de datele primite din backend. Medicul poate iniția autentificarea, poate naviga către lista pacienților, poate deschide fișa unui pacient, poate consulta istoricul măsurătorilor, poate configura pragurile de alertă și poate introduce recomandări. Datele răspuns sunt apoi afișate sub formă tabelară sau grafică.

Prelucrările realizate la nivelul interfeței web sunt centrate pe randarea și organizarea datelor primite, pe menținerea stării paginilor și pe validarea locală elementară a formularelor. Interfața nu stabilește singură dacă o alertă este validă și nu modifică structurile de date fără aprobarea logicii din backend.

Ieșirile interfeței web sunt cererile către API și prezentarea vizuală a rezultatelor către medic. Interacțiunile sunt, așadar, duble: cu medicul ca utilizator final și cu backend-ul ca furnizor al logicii centrale și al datelor persistente.

3.7. Baza de date

Baza de date PostgreSQL asigură persistența informațiilor sistemului și reprezintă punctul central de stocare pentru datele medicale, datele de autentificare și datele administrative. Alegerea unei baze de date relaționale este coerentă cu nevoia de integritate, trasabilitate și structurare clară a relațiilor dintre utilizatori, pacienți, măsurători, capturi ECG, praguri, alerte și recomandări.

Intrările bazei de date sunt interogările și operațiile de actualizare primite exclusiv din backend. Acestea includ inserarea măsurătorilor, salvarea alertelor, citirea pragurilor, consultarea istoricului și gestionarea datelor pacientului.

Prelucrările bazei de date constau în stocarea persistentă a datelor, aplicarea restricțiilor de integritate, menținerea relațiilor prin chei externe și optimizarea accesului prin indecși. Ea nu efectuează logică medicală și nu comunică cu alte componente în afara backend-ului.

Ieșirile bazei de date sunt seturile de rezultate și confirmările întoarse către backend. Interacțiunea acestei componente este, prin proiectare, exclusivă și strict controlată.

3.8. Modulul de management și acces administrativ

Modulul de management și acces administrativ este construit în jurul serviciului WireGuard și are rolul de a oferi un canal separat de acces către infrastructura internă. El nu face parte din traseul de utilizare normală al sistemului pentru pacient sau medic, dar este esențial pentru administrare, întreținere și depanare.

Intrările acestui modul sunt conexiunile administrative venite de la membrii echipei autorizați. Prelucrările constau în validarea și stabilirea tunelurilor VPN, precum și în rutarea traficului administrativ către containerele interne. Ieșirile sunt sesiunile de acces controlat către infrastructură.

Interacțiunile sunt limitate la administratorii sistemului și la componentele interne ale rețelei private. Avantajul principal al acestei componente este separarea completă dintre traficul de producție și cel de administrare, ceea ce contribuie la securitate și la controlul mai bun al mediului.

4. DESCRIEREA COMUNICĂRII ÎNTRE MODULE

4.1. Principii generale de comunicare

Comunicarea dintre componentele sistemului este proiectată astfel încât fiecare canal să fie clar delimitat prin sursă, destinație, protocol, tip de date și restricții. Această formalizare este necesară pentru ca implementarea să poată fi realizată în paralel, fără neclarități privind structura mesajelor și responsabilitatea fiecărei componente. În acord cu cerințele profesorului, această secțiune descrie în mod explicit relațiile dintre module și modul în care circulă informația în sistem.

În proiectarea actuală, comunicarea urmează o logică stratificată: datele pleacă de la hardware către aplicația mobilă, apoi către infrastructura centrală, sunt procesate de backend și stocate în baza de date. În sens invers, istoricul și datele de monitorizare sunt solicitate de aplicația mobilă sau interfața web și furnizate de backend pe baza informațiilor persistate.

4.2. Tabelul canalelor de comunicație

Sursă	Destinație	Tip canal	Protocol / suport	Tip de date	Observații
Senzori	Arduino	intern hardware	semnal analogic / digital	valori brute	flux local
Arduino	Aplicație mobilă	local fără fir	Bluetooth	date brute / secvențe	distanță scurtă
Aplicație mobilă	Nginx	extern securizat	HTTPS	JSON	pacient
Browser medic	Nginx	extern securizat	HTTPS	cereri interfață și API	medic
Nginx	Backend	intern	rutare HTTP	cereri API	rețea internă

Backend	PostgreSQL	intern	TCP/IP + SQL	interogări și actualizări	acces exclusiv
Administrator	WireGuard	administrativ	VPN securizat	trafic de management	separat de producție

Acest tabel sintetizează canalele principale de comunicație și oferă un reper clar pentru implementare. Fiecare dintre aceste canale este detaliat în subpunctele următoare.

4.3. Comunicarea dintre modulul inteligent și aplicația mobilă

Comunicarea dintre placa Arduino și aplicația mobilă este realizată prin Bluetooth și reprezintă primul canal de transfer efectiv al datelor în sistem. La acest nivel, aplicația mobilă acționează ca receptor al informațiilor produse de prototipul hardware. Se presupune că fluxul transmis de Arduino conține cel puțin tipul parametrului și valoarea sau secvența de valori asociată.

Pentru temperatură și puls, comunicarea poate fi tratată ca transmitere de valori individuale sau de serii scurte. Pentru ECG, este recomandată transmiterea unei capturi reprezentate printr-o secvență de eșantioane și metadate minime, precum momentul capturii și durata aproximativă. Proiectarea nu impune un protocol sofisticat la acest nivel, dar impune ca structura datelor să poată fi interpretată univoc de aplicația mobilă.

Un aspect important este că acest canal este mai vulnerabil la întreruperi locale decât cele din infrastructura centrală. Din acest motiv, aplicația mobilă trebuie să poată detecta lipsa de semnal sau întreruperea transmisiei și să gestioneze corespunzător situațiile în care fluxul este incomplet.

4.4. Comunicarea dintre aplicația mobilă și backend

Aplicația mobilă transmite datele către infrastructura centrală prin cereri HTTPS, iar formatul utilizat pentru schimbul de informații este JSON. Alegerea acestui format este justificată de simplitatea lui, de integrarea naturală cu backend-ul ASP.NET Core și de ușurința cu care poate fi folosit atât pentru măsurători simple, cât și pentru capturi mai complexe.

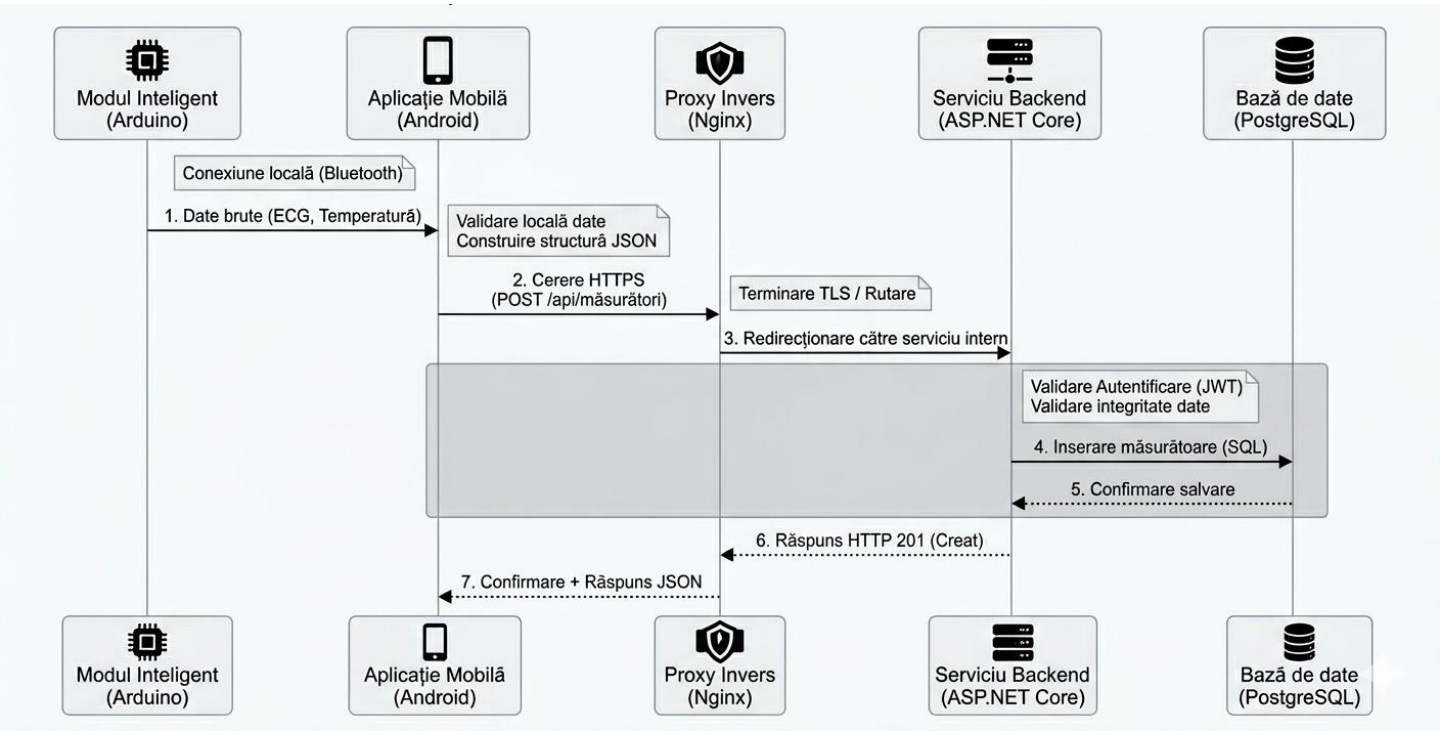
Pentru transmiterea unei măsurători simple, se propune o structură de tipul:

```
{
 "idPacient": 101,
 "tipParametru": "temperatura",
 "valoare": 36.7,
 "unitate": "C",
 "dataOra": "2026-06-10T14:25:00",
 "sursa": "aplicatieMobila"
}
```

Pentru o captură ECG, structura trebuie extinsă pentru a include secvența de valori:

```
{
 "idPacient": 101,
 "tipParametru": "ecg",
 "secventaValori": [512, 514, 517, 520, 516, 510],
 "durataCapturiiSec": 5,
 "dataOra": "2026-06-10T14:25:00",
 "sursa": "aplicatieMobila"
}
```


Backend-ul validează aceste date înainte de stocare. Sunt verificate identitatea utilizatorului, existența pacientului, corectitudinea structurii JSON, coerența dintre tipul parametrului și forma datelor, precum și plauzibilitatea valorilor. În cazul în care aceste condiții nu sunt îndeplinite, cererea este respinsă.



În plus, dacă aplicația mobilă nu poate transmite imediat datele din cauza lipsei conexiunii, trebuie să poată stoca local temporar datele și să le retransmită ulterior. Această logică nu modifică structura canalului de comunicație, dar influențează comportamentul clientului la apariția erorilor.

4.5. Comunicarea dintre interfața web și backend

Interfața web comunică cu backend-ul prin puncte de acces API, utilizând HTTPS și răspunsuri structurate în JSON. Spre deosebire de aplicația mobilă, care transmite și date noi colectate de la senzori, interfața web este orientată mai ales spre consultare și administrare.

Principalele categorii de cereri sunt autentificarea medicului, încărcarea listei pacienților, încărcarea fișei unui pacient, solicitarea istoricului măsurătorilor, consultarea alertelor și adăugarea de recomandări sau praguri.

Toate aceste operații trebuie să fie filtrate prin verificarea rolului și a drepturilor utilizatorului. Un medic trebuie să poată accesa doar datele pacienților asociați lui.

Pentru claritatea implementării, este utilă organizarea punctelor de acces în categorii distincte. Un tabel orientativ poate fi folosit în document pentru a susține această proiectare:

Metodă	Rută propusă	Scop
POST	/api/autentificare/login-medic	autentificare medic
POST	/api/autentificare/login-pacient	autentificare pacient
GET	/api/pacienti	încărcare listă pacienți
GET	/api/pacienti/{id}	încărcare fișă pacient
GET	/api/pacienti/{id}/istoric	istoric măsurători
GET	/api/pacienti/{id}/alerte	istoric alerte
POST	/api/pacienti/{id}/praguri	configurare praguri
POST	/api/pacienti/{id}/recomandari	salvare recomandare

(Rutele propuse au caracter orientativ și pot să difere în etapa de implementare, în funcție de structura finală a controllerelor backend)

4.6. Comunicarea dintre backend și baza de date

Comunicarea dintre backend și PostgreSQL este internă și se realizează prin interogări SQL peste TCP/IP în rețeaua privată. Acest canal este unul dintre cele mai importante din punct de vedere al proiectării, deoarece aici se materializează persistența datelor și relațiile dintre entități.

Backend-ul utilizează această comunicație pentru:

- salvarea măsurătorilor;
- salvarea capturilor ECG;
- citirea pragurilor de alertă;
- citirea istoricului recent necesar verificării persistenței unei abateri;
- inserarea alertelor;
- încărcarea fișelor și recomandărilor;
- generarea istoricului și a rapoartelor.

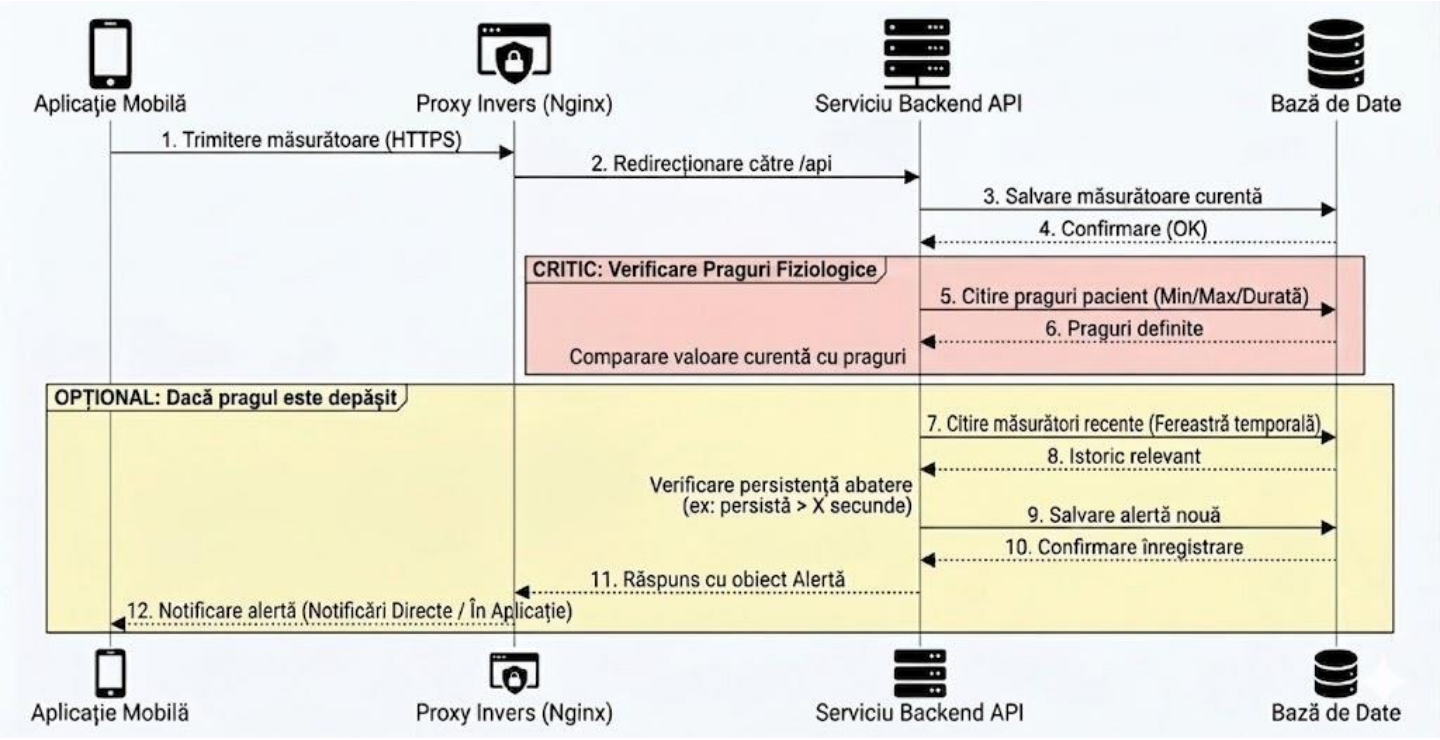
Nicio altă componentă nu trebuie să comunice direct cu baza de date. Această restricție este o decizie de proiectare esențială, deoarece menține controlul logicii și reduce riscul de inconsistență între module.

4.7. Gestionarea erorilor și a răspunsurilor

În proiectarea canalelor de comunicație este important să se stabilească și modul de tratare a erorilor. Backend-ul trebuie să întoarcă răspunsuri standardizate, atât din punct de vedere al codurilor de stare, cât și din punct de vedere al mesajelor explicative.

Cod	Semnificație	Utilizare tipică
200	operație reușită	citire date
201	resursă creată	inserare măsurătoare, alertă, recomandare
400	cerere invalidă	structură greșită, date incomplete
401	neautorizat	autentificare invalidă
403	acces interzis	lipsă drepturi
404	resursă inexistentă	pacient negăsit
500	eroare internă	problemă neprevăzută

În cazul aplicației mobile trebuie făcută distincția între erorile logice și erorile de rețea. O eroare logică, cum ar fi o cerere invalidă, nu trebuie retransmisă automat. În schimb, o eroare de rețea poate justifica stocarea temporară și o nouă încercare de transmitere. În cazul generării alertelor, este important ca backend-ul să întoarcă un răspuns coerent atunci când alerta a fost creată, astfel încât aplicația mobilă sau interfața web să poată reacționa corespunzător.



5. STRUCTURI DE BAZE DE DATE ȘI FIȘIERE

5.1. Principii de proiectare a bazei de date

Modelul bazei de date trebuie să reflecte structura logică a sistemului și să permită stocarea clară a tuturor informațiilor relevante fără redundanțe inutile. Din punct de vedere conceptual, sistemul are nevoie de entități distincte pentru utilizatori, medici, pacienți, dispozitive, măsurători, capturi ECG, praguri, recomandări și alerte.

Separarea acestor entități este justificată atât de diferențele de semnificație dintre ele, cât și de nevoia de trasabilitate și întreținere.

În proiectarea de față, modelul relațional trebuie să permită:

- autentificarea separată a medicului și pacientului;
- asocierea fiecărui pacient cu un medic;
- stocarea repetată în timp a măsurătorilor;
- stocarea distinctă a capturilor ECG;
- definirea pragurilor individuale pentru fiecare pacient și parametru;
- generarea și urmărirea alertelor în timp;
- evidența recomandărilor medicale.

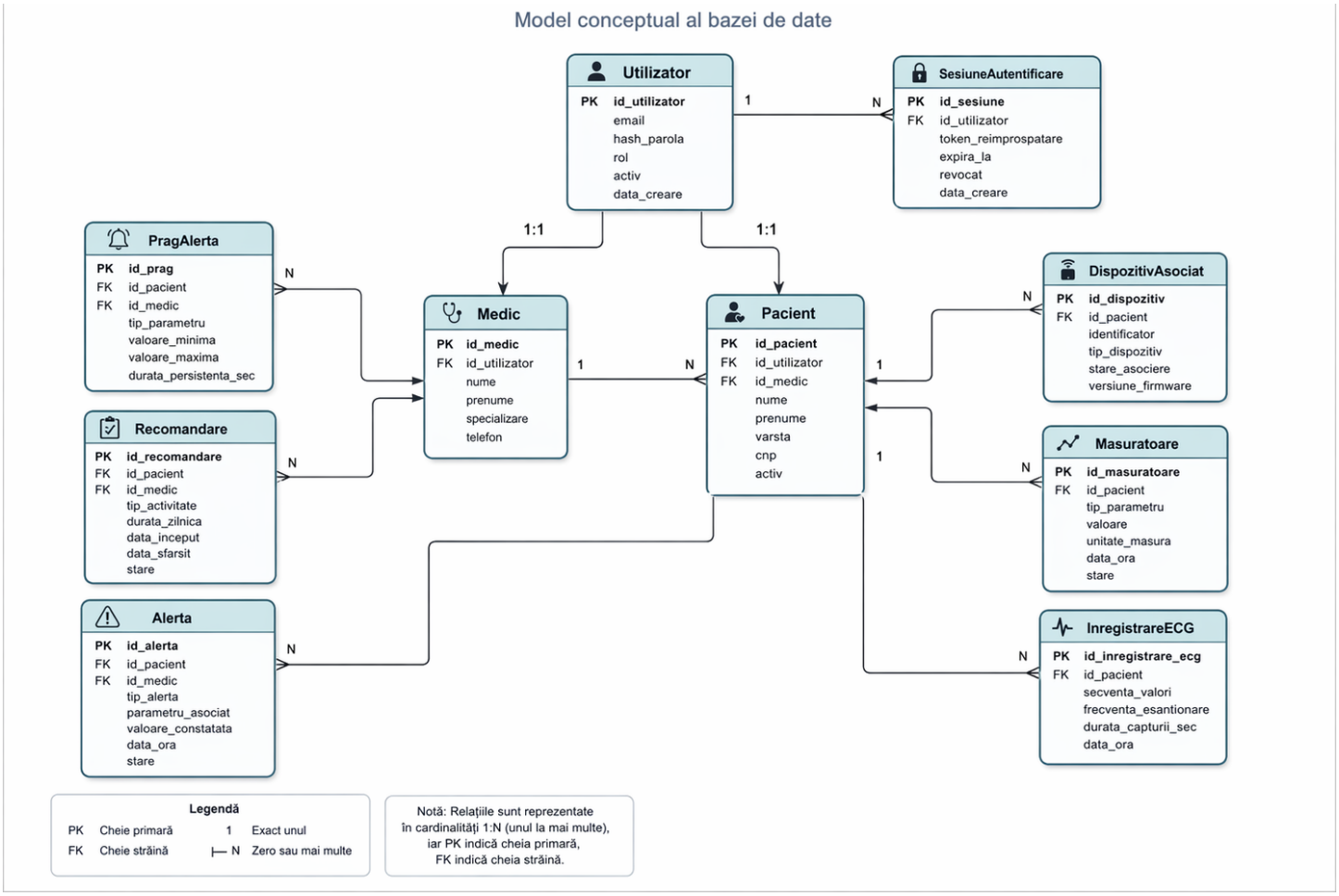
5.2. Schema relațională generală și diagrama de relații între entități

Entitățile principale ale modelului de date sunt:

- Utilizator;
- Medic;
- Pacient;
- DispozitivAsociat;
- PragAlerta;
- Masuratoare;
- InregistrareECG;
- Recomandare;
- Alerta;
- SesiuneAutentificare.

Relațiile de bază sunt următoarele:

- un utilizator corespunde unui medic sau unui pacient;
- un medic poate avea mai mulți pacienți;
- un pacient poate avea mai multe măsurători, mai multe capturi ECG, mai multe praguri, mai multe recomandări și mai multe alerte;
- un pacient poate avea dispozitive asociate;
- alertele și recomandările păstrează legătura atât cu pacientul, cât și cu medicul.



5.3. Definirea tabelelor

Tabelul Utilizator:
Acest tabel stochează informațiile de autentificare și rolul utilizatorului în sistem.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_utilizator	integer / serial	—	identificator unic	cheie primară
email	varchar	150	adresă de autentificare	NOT NULL, UNIQUE
hash_parola	varchar	255	parolă stocată securizat	NOT NULL
rol	varchar	20	rol utilizator	NOT NULL
activ	boolean	—	cont activ/inactiv	NOT NULL
data_creare	timestamp	—	momentul creării	NOT NULL

Tabelul Medic:
Acest tabel conține informațiile de profil pentru utilizatorii cu rol de medic.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_medic	integer / serial	—	identificator medic	cheie primară
id_utilizator	integer	—	legătură cu autentificarea	FOREIGN KEY, UNIQUE
nume	varchar	100	nume	NOT NULL

prenume	varchar	100	prenume	NOT NULL
specializare	varchar	100	specializare	NULL permis
telefon	varchar	20	telefon	NULL permis

Tabelul Pacient:

Acest tabel conține informațiile de profil și context medical pentru pacient.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_pacient	integer / serial	—	identificator pacient	cheie primară
id_utilizator	integer	—	contul pacientului	FOREIGN KEY, UNIQUE
id_medic	integer	—	medic responsabil	FOREIGN KEY, NOT NULL
nume	varchar	100	nume	NOT NULL
prenume	varchar	100	prenume	NOT NULL
varsta	integer	—	vârstă	CHECK > 0
cnp	varchar	13	identificator personal	UNIQUE
adresa	varchar	255	adresă	NULL permis
telefon	varchar	20	telefon	NULL permis
email	varchar	150	e-mail	NULL permis
profesie	varchar	100	profesie	NULL permis
loc_munca	varchar	150	loc de muncă	NULL permis
istoric_medical	text	—	istoric medical	NULL permis
alergii	text	—	alergii	NULL permis
consultatii_cardiologice	text	—	observații cardiologice	NULL permis
activ	boolean	—	stare activă	NOT NULL

Tabelul DispozitivAsociat:

Acest tabel păstrează relația dintre pacient și dispozitivul utilizat.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_dispozitiv	integer / serial	—	identificator dispozitiv	cheie primară
id_pacient	integer	—	pacient asociat	FOREIGN KEY, NOT NULL
identificator	varchar	100	identificator hardware	UNIQUE, NOT NULL
tip_dispozitiv	varchar	50	tip dispozitiv	NOT NULL
stare_asociere	varchar	20	stare asociere	NOT NULL
versiune_firmware	varchar	50	versiune internă	NULL permis
data_asociere	timestamp	—	moment asociere	NOT NULL

Tabelul PragAlerta:

Acest tabel definește limitele normale pentru fiecare pacient și parametru, precum și durata minimă de persistență.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_prag	integer / serial	—	identificator prag	cheie primară
id_pacient	integer	—	pacient asociat	FOREIGN KEY, NOT NULL

id_medic	integer	—	medic configurator	FOREIGN KEY, NOT NULL
tip_parametru	varchar	30	parametru urmărit	NOT NULL
valoare_minima	numeric	—	limită inferioară	NULL permis
valoare_maxima	numeric	—	limită superioară	NULL permis
durata_persistenta_sec	integer	—	durată minimă	NOT NULL
activ	boolean	—	prag activ	NOT NULL

Tabelul Masuratoare:

Acest tabel reține măsurătorile simple, precum temperatură sau puls.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_masuratoare	integer / serial	—	identificator măsurătoare	cheie primară
id_pacient	integer	—	pacient asociat	FOREIGN KEY, NOT NULL
tip_parametru	varchar	30	tip parametru	NOT NULL
valoare	numeric	—	valoare măsurată	NOT NULL
unitate_masura	varchar	20	unitate	NOT NULL
data_ora	timestamp	—	moment măsurare	NOT NULL
stare	varchar	20	stare validare	NOT NULL
sursa	varchar	30	sursa măsurătorii	NOT NULL
observatii	varchar	255	observații	NULL permis

Tabelul InregistrareECG:

Acest tabel reține capturile ECG, separate de măsurătorile simple.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_inregistrare_ecg	integer / serial	—	identificator captură	cheie primară
id_pacient	integer	—	pacient asociat	FOREIGN KEY, NOT NULL
secventa_valori	text	—	secvență de valori	NOT NULL
frecventa_esantionare	integer	—	frecvență de eșantionare	NULL permis
durata_capturii_sec	integer	—	durată captură	NOT NULL
data_ora	timestamp	—	moment captură	NOT NULL
observatii	text	—	observații	NULL permis
stare_validare	varchar	20	calitate / validare	NOT NULL

Tabelul Recomandare:

Acest tabel conține recomandările emise de medic pentru pacient.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_recomandare	integer / serial	—	identificator recomandare	cheie primară
id_pacient	integer	—	pacient vizat	FOREIGN KEY, NOT NULL
id_medic	integer	—	medic emitent	FOREIGN KEY, NOT NULL
tip_activitate	varchar	100	activitate recomandată	NOT NULL

durata_zilnica	varchar	50	durată recomandată	NULL permis
alte_indicatii	text	—	indicații suplimentare	NULL permis
data_inceput	date	—	început recomandare	NOT NULL
data_sfarsit	date	—	sfârșit recomandare	NULL permis
stare	varchar	20	stare recomandare	NOT NULL

Tabelul Alerta:

Acest tabel conține evenimentele generate automat atunci când o abatere persistă peste pragurile definite.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_alerta	integer / serial	—	identificator alertă	cheie primară
id_pacient	integer	—	pacient vizat	FOREIGN KEY, NOT NULL
id_medic	integer	—	medic asociat	FOREIGN KEY, NOT NULL
tip_alerta	varchar	30	tip alertă	NOT NULL
parametru_asociat	varchar	30	parametru care a generat alerta	NOT NULL
prag_minim	numeric	—	limită inferioară folosită	NULL permis
prag_maxim	numeric	—	limită superioară folosită	NULL permis
valoare_constatata	numeric	—	valoare observată	NOT NULL
data_ora	timestamp	—	moment alertă	NOT NULL
durata_persistenta_sec	integer	—	durată de confirmare	NOT NULL
mesaj	varchar	255	mesaj alertă	NOT NULL
stare	varchar	20	stare alertă	NOT NULL

Tabelul SesiuneAutentificare:

Acest tabel poate fi utilizat pentru gestionarea tokenurilor de sesiune sau de reîmprospătare.

Câmp	Tip propus	Lungime	Semnificație	Restricții
id_sesiune	integer / serial	—	identificator sesiune	cheie primară
id_utilizator	integer	—	utilizator asociat	FOREIGN KEY, NOT NULL
token_reimprospatare	varchar	255	token	UNIQUE
expira_la	timestamp	—	expirare	NOT NULL
revocat	boolean	—	token revocat	NOT NULL
data_creare	timestamp	—	creare sesiune	NOT NULL

5.4. Relații, chei, indecși și restricții

Modelul propus urmărește o organizare clară și relațională a datelor. Relațiile principale sunt de tip 1:1 între Utilizator și Medic sau Pacient și de tip 1:N între Medic și Pacient, respectiv între Pacient și toate entitățile medicale sau operaționale asociate.

Pentru performanță și integritate se recomandă:

- definirea cheilor primare pentru fiecare tabel;
- definirea cheilor externe pentru relațiile dintre entități;
- aplicarea restricțiilor NOT NULL acolo unde lipsa valorii ar face datele inutilizabile;
- aplicarea restricțiilor UNIQUE pentru câmpurile de identificare logică;
- aplicarea unor indecși pe câmpurile folosite frecvent la interogare.

Indecși utili:

- email în Utilizator;
- id_medic în Pacient;
- id_pacient și data_ora în Masuratoare;
- id_pacient și data_ora în Alerta;
- id_pacient și tip_parametru în PragAlerta.

5.5. Fișiere auxiliare și structuri temporare

Pe lângă persistența principală în baza de date, sistemul poate utiliza o serie de fișiere auxiliare necesare funcționării infrastructurii și rezilienței aplicației. Acestea includ fișiere de configurare pentru Nginx, backend, frontend și WireGuard, fișiere de jurnal pentru infrastructură și, la nivelul aplicației mobile, structuri temporare locale pentru păstrarea datelor care nu au putut fi transmise imediat din cauza lipsei conexiunii.

Aceste fișiere nu înlocuiesc persistența principală a datelor medicale și nu reprezintă sursa de adevăr a sistemului, ci doar componente auxiliare necesare funcționării sale corecte.

Prin descrierea arhitecturii programului, a componentelor, a relațiilor dintre module și a structurilor de date persistente, documentul de față oferă baza necesară pentru etapa de codificare și pentru continuarea dezvoltării produsului într-o manieră tehnică unitară. În acest sens, proiectarea realizată susține evoluția sistemului din faza de documentare și prototipare către o aplicație funcțională, extensibilă și adaptabilă cerințelor reale ale monitorizării medicale la distanță.